

TALLER DOM / JAVASCRIPT

JIMENA SOFÍA VALIENTE BLANDÓN

Ejercicio 1: El escultor de títulos.

Enunciado: Crea una página HTML con tres etiquetas <h1>, <h2> y <h3>. Utilizando JavaScript selecciona el <h2>, cambia su contenido por tu nombre completo y modifica su color a azul y su alineación al centro.

Antes de cambiar el h2:

Título interesante

Subtítulo interesante

Prometo no soltar tu mando cuando sintamos sus ojos sobre nosotras

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio1 | Nombre</title>
</head>
<body>
  <h1>Título interesante</h1>
  <h2>Subtítulo interesante</h2>
  <h3>Prometo no soltar tu mando cuando sintamos sus ojos sobre
nosotras</h3>

  <script src = "ejercicio1.js"></script>
</body>
</html>
```

Después de cambiar el h2:

Titulo interesante

Jimena Sofia Valiente Blandón

Prometo no soltar tu mando cuando sintamos sus ojos sobre nosotras

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio1 | Nombre</title>
</head>
<body>
  <h1>Titulo interesante</h1>
  <h2 id = "cambiar">Subtitulo interesante</h2>
  <h3>Prometo no soltar tu mando cuando sintamos sus ojos sobre
nosotras</h3>

  <script>
    const $cambiar = document.getElementById('cambiar');
    $cambiar.textContent = 'Jimena Sofía Valiente Blandón';
    $cambiar.style.color = "blue";
    $cambiar.style.textAlign = 'center';
  </script>
</body>
</html>
```

Preguntas de validación:

1. ¿Qué método utilizaste para seleccionar el elemento por su etiqueta?

RTA// Utilicé el `.getElementById`, en el que en el h2 le puse la ID de “cambiar” para poder identificarlo en el script y así poder realizarle los cambios.

2. ¿Cuál es la diferencia entre `.innerText` y `.innerHTML` en este ejercicio?

RTA// La diferencia entre ellos dos radica en que el `.innerText` extrae únicamente el texto visible, ignorando todas las etiquetas HTML y respetando los estilos CSS. Por otro lado el `.innerHTML` obtiene o modifica el contenido HTML interno, es decir, que este sí incluye las etiquetas que se parsean y renderizan en el DOM.

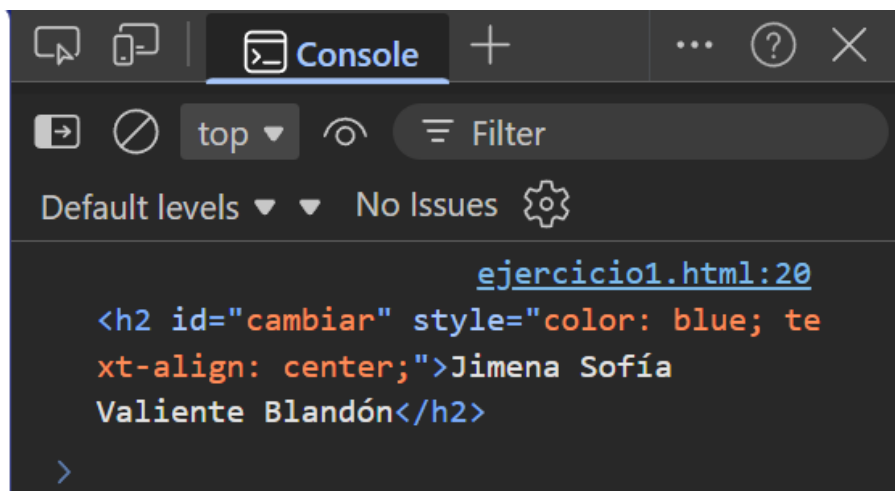
3. ¿Qué propiedad del objeto `.style` permite cambiar el color?

RTA// La propiedad es `.style.color = "color"`, en la que antes de `style` está el nombre del elemento al cual se le quiere aplicar el color, en el caso de mi código sería `$cambiar.style.color = "blue"`; para así aplicarle el color azul al texto.

```
$cambiar.style.color = "blue";
```

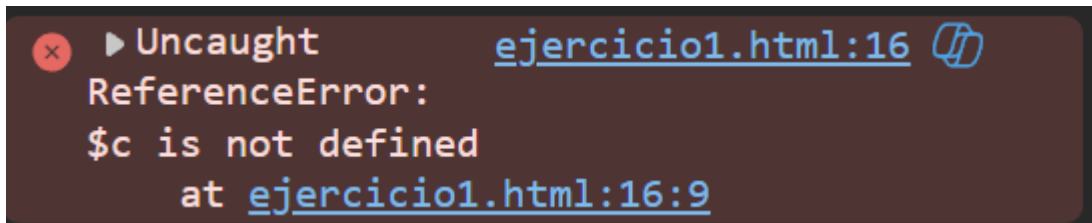
4. ¿Cómo verificarías en la consola si el elemento fue seleccionado correctamente?

RTA// Escribiendo `"console.log($cambiar)"` y verificar en la consola que el elemento existe y muestra su objeto en el DOM.



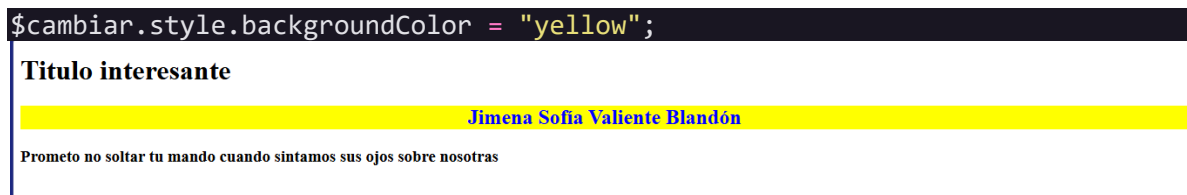
5. ¿Qué sucede si intentas seleccionar un elemento que no existe?

RTA// Devuelve **null**, sin error pero las operaciones posteriores que se quieran realizar fallan.



6. ¿Cómo cambiarías el color de fondo (background-color) mediante JS?

RTA// En el caso de mi código sería:



7. ¿Cuál es el objeto global que contiene al document?

RTA// “**window**” es el objeto global principal en JavaScript dentro de un navegador, ya que este es la raíz en donde se ejecuta todo el código, y en este caso “**document**” sería una de sus propiedades.

Para verlo en consola en el código se escribe “**console.log(window);**”.

8. ¿Es posible seleccionar varios elementos a la vez con getElementById?

RTA// No, no es posible, este devolverá un solo elemento, ya que los IDs deben ser únicos.

9. ¿Qué comando usarías para ver todas las propiedades CSS aplicadas al elemento en la consola?

RTA// Utilizaría el comando

“console.log(getComputedStyle(\$cambiar))”

```
ejercicio1.html:21  
<h2 id="cambiar" style="color: blue; text-align: center; background-color: yellow;">  
Jimena Sofía Valiente Blandón</h2>
```

```
ejercicio1.html:22  
▶ CSSStyleDeclaration {0: 'accent-color', 1: 'align-content', 2: 'align-items', 3: 'align-self', 4: 'alignment-baseline', 5: 'anchor-name', 6: 'anchor-scope', 7: 'animation-composition', 8: 'animation-delay', 9: 'animation-direction', 10: 'animation-duration', 11: 'animation-fill-mode', 12: 'animation-iteration-count', 13: 'animation-name', 14: 'animation-play-state', 15: 'animation-range-end', 16: 'animation-range-start', 17: 'animation-timeline', 18: 'animation-timing-function', 19: 'app-region', 20: 'appearance', 21: 'backdrop-filter', 22: 'backface-visibility', 23: 'background-attachment', 24: 'background-blend-mode', 25: 'background-clip', 26: 'background-color', 27: 'background-image', 28: 'background-origin', 29: 'background-position', 30: 'background-repeat', 31: 'background-size', 32: 'baseline-shift', 33: 'baseline-source', 34: 'block-size', 35: 'border-block-end-color', 36: 'border-block-end-style', 37: 'border-block-end-width', 38: 'border-block-start-color', 39: 'border-block-start-style', 40: 'border-block-start-width', 41: 'border-bottom-color', 42: 'border-bottom-left-radius', 43: 'border-bottom-right-radius', 44: 'border-bottom-style', 45: 'border-bottom-width', 46: 'border-collapse', 47: 'border-end-end-radius', 48: 'border-end-start-radius', 49: 'border-image-outset', 50: 'border-image-repeat', 51: 'border-image-slice', 52: 'border-image-source', 53: 'border-image-width', 54: 'border-inline-end-color', 55: 'border-inline-end-style', 56: 'border-inline-end-width', 57: 'border-inline-start-color', 58: 'border-inline-start-style', 59: 'border-inline-start-width', 60: 'border-left-color', 61: 'border-left-style', 62: 'border-left-width', 63: 'border-right-color', 64: 'border-right-style', 65: 'border-right-width', 66: 'border-start-end-radius', 67: 'border-start-start-radius', 68: 'border-top-color', 69: 'border-top-left-radius', 70: 'border-top-right-radius', 71: 'border-top-style', 72: 'border-top-width', 73: 'bottom', 74: 'box-decoration-break', 75: 'box-shadow', 76: 'box-sizing', 77: 'break-after', 78: 'break-before', 79: 'break-inside', 80: 'buffered-rendering', 81: 'caption-side', 82: 'caret-animation', 83: 'caret-color', 84: 'caret-shape', 85: 'clear', 86: 'clip', 87: 'clip-path', 88: 'clip-rule', 89: 'color', 90: 'color-interpolation', 91: 'color-interpolation-filters', 92: 'color-rendering', 93: 'column-count', 94: 'column-gap', 95: 'column-height', 96: 'column-rule-color', 97: 'column-rule-style', 98: 'column-rule-width', 99: 'column-span', ...}
```

10. ¿Cómo podrías deshacer el cambio de color usando código?

RTA// Podría hacer `$cambiar.style.color = ""` o
`$cambiar.style.color = 'inherit'`

```
$cambiar.style.color = 'inherit';
```

Título interesante

Jimena Sofia Valiente Blandón

Prometo no soltar tu mando cuando sintamos sus ojos sobre nosotras

Ejercicio 2: El gestor de Listas Dinámicas.

Enunciado: Crea una lista vacía (). Diseña un script que, al cargar la página, inserte 5 elementos de lista () numerados del 1 al 5. El número 3 debe tener un estilo de negrita aplicado mediante una clase de CSS añadida por JS.

Ejercicio 2

- Elemento 1
- Elemento 2
- **Elemento 3**
- Elemento 4
- Elemento 5

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel = "stylesheet" href = "ejercicio2.css">
  <title>Ejercicio 2 | lista</title>
</head>
<body>
  <h1>Ejercicio 2</h1>
  <ul id = "lista"></ul>

  <script src = "ejercicio2.js"></script>
</body>
</html>
```

Css

```
.resaltado{
  font-weight: bold;
  color: #2c3e50;
}
```

Js

```
const $lista = document.getElementById("lista");

for (let i = 1; i<=5; i++) {
  const $item = document.createElement("li");
  $item.textContent = `Elemento ${i}`;

  if (i === 3) {
    $item.classList.add('resaltado');
  }

  $lista.appendChild($item);
}
```

Preguntas de validación:

1. ¿Qué método usaste para crear el nuevo nodo ?

RTA// Utilicé el método **document.createElement("li")**;. Este lo que hace es crear el elemento en la memoria del navegador antes de ser insertado en la página.

2. ¿Cómo vinculaste el nuevo elemento como hijo del ?

RTA// Lo vinculé con el método **\$lista.appendChild(\$item)**, que agrega el nuevo nodo al final de la lista de hijos del elemento padre.

3. ¿Qué estructura de control (bucle) utilizaste para la repetición?

RTA// El bucle **for** con un contador desde 1 hasta 5.

For (let i = 0; i <=5; i++)

4. ¿Qué método de JS permite añadir una clase de CSS a un elemento?

RTA// El método **classList.add("clase")**. Este es preferible usarlo en vez de **className** porque no sobrescribe las clases que ya existen.

5. ¿Cuál es la diferencia entre `appendChild()` y `prepend()`?

RTA// `appendChild()` lo que haces es insertar el elemento al final de la lista de hijos, mientras que `prepend()` lo inserta al inicio.

6. ¿Cómo podrías borrar el segundo elemento de la lista mediante código?

RTA// Seleccionando el elemento por su índice que en este caso sería `[1]` ya que estos empiezan en `0` y aplicar el `remove()`:

```
$lista.children[1].remove();
```

7. ¿Qué propiedad permite cambiar solo el texto de un nodo sin afectar el HTML?

RTA// La propiedad **`textContent`**, ya que este trata todo lo que se le pase como texto literal, incluso si se intentan pasar etiquetas, a diferencia del `innerHTML`.

8. ¿Cómo identificarías el elemento “padre” de un `<li>` seleccionado?

RTA// Usando la propiedad **`$item.parentElement`**.

9. ¿Se pueden añadir atributos personalizados (como `data-id`) al crear el elemento?

RTA// Sí, usando el método **`setAttribute('data-id', 'valor')`**.

10. ¿Qué ventaja tiene usar `createElement` sobre escribir el texto plano en `innerHTML`?

RTA// Es más seguro ya que este puede evitar ataques XSS y es eficiente en términos de rendimiento, ya que no obliga al navegador a parsear todo el HTML del contenedor cada vez que se agrega un elemento; solo manipula el nodo específico.

Ejercicio 3: El sensor de interacción.

Enunciado: Inserta una imagen en tu HTML. Configura dos eventos: uno para que cuando el mouse pase sobre la imagen esta se agrande (escala 1:2) y otro para que cuando el mouse salga, vuelva a su tamaño original.

Imagen normal



Imagen agrandada



Html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel = "stylesheet" href = "ejercicio3.css">
  <title>Ejercicio 3 | Imagen</title>
</head>
<body>
  

  <script src = "ejercicio3.js"></script>
</body>
</html>
```

Css:

```
#miImagen {
  width: 300px;
  transition: transform 0.3s ease; /* Para que el escalado sea suave */
  cursor: pointer;
}
```

Js:

```
const $img = document.getElementById('miImagen');

// Evento de entrada
$img.addEventListener('mouseenter', () => {
  $img.style.transform = "scale(1.2)";
});

// Evento de salida
$img.addEventListener('mouseleave', () => {
  $img.style.transform = "scale(1)";
});
```

Preguntas de validación:

1. ¿Cuál es el nombre del evento que detecta cuando el puntero entra al elemento?

RTA// Se puede usar **mouseenter** o **mouseover**, en este caso utilicé **mouseenter** ya que este no se activa al pasar sobre elementos hijos y es ideal para efectos visuales como en este ejercicio.

2. ¿Qué método recomiendas para asignar el evento: **onmouseover** o **addEventListener**?

RTA// **addEventListener**.

3. ¿Por qué es mejor práctica usar **addEventListener**?

RTA// Porque permite añadir múltiples funciones al un mismo evento sin sobrescribir las anteriores, separa mejor la lógica del HTML y ofrece mayor control.

4. ¿Cómo accedes al elemento que disparó el evento dentro de la función (palabra clave)?

RTA// Se utiliza la palabra clave **this** en funciones tradicionales o la propiedad **target** del objeto de evento.

5. ¿Qué propiedad de CSS transformaste para el escalado?

RTA// La propiedad **transform**, específicamente utilizando la función **scale()** para hacer que la imagen se agrande o disminuya de tamaño según la escala que se solicitó.

6. ¿Cómo evitarías que el evento se dispare más de una vez?

RTA// Al usar **addEventListener**, se puede pasar un tercer argumento opcional **{once: true}**.

7. ¿Cuál es el nombre del evento para cuando el mouse sale del elemento?

RTA// El evento se llama **mouseleave**.

8. ¿Podrías cambiar la fuente de la imagen (src) durante el evento de entrada?

RTA// Sí, mediante la instrucción `$img.src = "ruta_de_la_nueva_imagen";`. Esta es una técnica común para efectos e hover.

9. ¿Cómo harías para que el cambio de tamaño sea suave (transición)?

RTA// Añadiendo la propiedad **transition** en el CSS del elemento. Aunque tambien se podría hacer por JS, pero sigue siendo más eficiente hacerlo por CSS.

10. ¿Qué objeto del navegador contiene la información de las coordenadas del mouse?

RTA// El objeto **MouseEvent**, que se recibe automáticamente como argumento en la función del evento. Este contiene propiedades como **clientX** y **clientY**.

Ejercicio 4: El validador de Formularios en Vivo.

Enunciado: Crea un campo de texto (input) y un párrafo vacío debajo. Mientras el usuario escribe, el párrafo debe mostrar en tiempo real cuántos caracteres lleva escritos. Si supera los 20 caracteres, el borde del input debe ponerse rojo.

prometo no soltar tu mano

Caracteres: 25

Html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel = "stylesheet" href = "ejercicio4.css">
  <title>Ejercicio 4 | Caracteres</title>
</head>
<body>
  <input type="text" id="miInput" placeholder="Escribe algo..."
  class="normal">
  <p id="contador">Caracteres: 0</p>

  <script src = "ejercicio4.js"></script>
</body>
</html>
```

Css:

```
#miInput {
  padding: 10px;
  outline: none;
```

```

        transition: border 0.2s;
    }

    .limite-superado {
        border: 2px solid red;
    }

    .normal {
        border: 1px solid #ccc;
    }
}

```

Js:

```

const $input = document.getElementById('miInput');
const $contador = document.getElementById('contador');

// Foco automático al cargar
window.onload = () => $input.focus();

$input.addEventListener('input', (e) => {
    const longitud = e.target.value.length;

    // Actualizar el texto del contador
    $contador.textContent = `Caracteres: ${longitud}`;

    // Validar límite de 20 para el borde rojo
    if (longitud > 20) {
        $input.style.border = "2px solid red";
    } else {
        $input.style.border = "1px solid #ccc";
    }

    // Limpiar si llega a 30 (Pregunta 7)
    if (longitud >= 30) {
        $input.value = "";
        $contador.textContent = "Caracteres: 0";
    }
});

```

Preguntas de validación:

1. ¿Qué evento capturas para detectar cada pulsación de tecla o cambio en el input?

RTA// El evento **input**. Es el método más adecuado porque se dispara inmediatamente cada vez que el valor del elemento cambia.

2. ¿Cómo obtienes el valor actual de lo que el usuario ha escrito?

RTA// Mediante la propiedad **.value** del elemento.

3. ¿Qué propiedad de cadena (string) usaste para contar los caracteres?

RTA// La propiedad **.length**, que devuelve el número de caracteres que estén en el string.

4. ¿Cómo accedes a la propiedad del borde del elemento desde JS?

RTA// A través de **\$input.style.border**.

5. ¿Qué condicional (if/else) aplicaste para el cambio de color?

RTA// Un **if(longitud>20)** aplicar el borde rojo, y un **else** para restablecer el color original cuando la cantidad de caracteres vuelve a ser menor o igual a 20.

6. ¿Cuál es la diferencia entre los eventos input, change y keyup?

RTA// **input** se dispara con cualquier cambio de valor inmediato; **change** solo se dispara cuando el usuario sale del campo tras haber modificado el valor y **keyup** solo se dispara cuando se suelta una tecla.

7. ¿Cómo limpiarías el campo de texto automáticamente al llegar a 30 caracteres?

RTA// Evaluando la longitud en el evento y asignando una cadena vacía:

```
If ( longitud>=30){ $input.value = "";} 
```

8. ¿Se puede deshabilitar el input (disabled) mediante el DOM?

RTA// Sí, cambiando la propiedad booleana a true: **\$input.disabled = true;**

9. ¿Cómo evitarías que el usuario pegue texto (evento paste)?

RTA// Escuchando el evento **paste** y cancelando su acción:

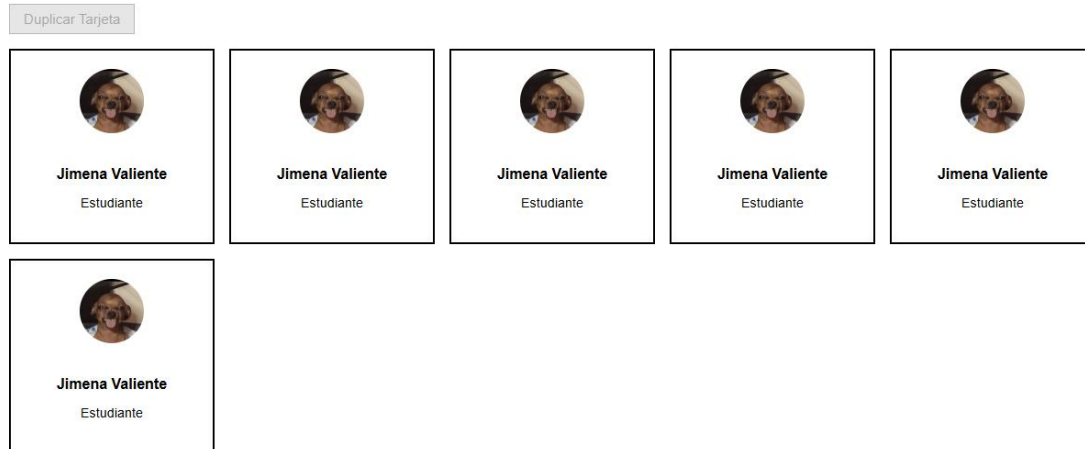
```
$input.addEventListener('paste', (e) => e.preventDefault()); .
```

10. ¿Qué método usarías para que el cursor se posicione automáticamente en el input al cargar la página?

RTA// El método **\$input.focus()**.

Ejercicio 5: El Clonador de Nodos.

Enunciado: Crea un div con una tarjeta de presentación (foto, nombre y cargo). Agrega un botón que, cada vez que se presione, duplique exactamente esa tarjeta debajo de la original.



Html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel = "stylesheet" href = "ejercicio5.css">
  <title>Ejercicio5 | Clonar</title>
</head>
<body>
  <button id="btn-clonar">Duplicar Tarjeta</button>

  <div id="contenedor-tarjetas">
    <div class="tarjeta" id="tarjeta-original">
      
      <p class="nombre">Jimena Valiente</p>
      <p class="cargo">Estudiante</p>
    </div>
  </div>

  <script src = "ejercicio5.js"></script>
</body>
```

```
</html>
```

Css:

```
body {  
  font-family: Arial, sans-serif;  
  padding: 20px;  
}
```

```
#contenedor-tarjetas {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 16px;  
  margin-top: 16px;  
}
```

```
.tarjeta {  
  border: 2px solid #000;  
  width: 180px;  
  padding: 20px;  
  text-align: center;  
}
```

```
.foto {  
  width: 70px;  
  height: 70px;  
  border-radius: 50%;  
  object-fit: cover;  
  background-color: #ccc;  
  margin: 0 auto 16px auto;  
}
```

```
.nombre {  
  font-weight: bold;  
  font-size: 1rem;  
  margin-bottom: 8px;  
}
```

```
.cargo {  
  font-size: 0.9rem;  
}
```

```
button {  
  padding: 6px 14px;
```

```
font-size: 0.95rem;  
cursor: pointer;  
}
```

Js:

```
const contenedor = document.getElementById('contenedor-tarjetas');  
const tarjetaOriginal = document.getElementById('tarjeta-original');  
const btnClonar = document.getElementById('btn-clonar');  
const MAX_CLONES = 5;  
  
let numClones = 0;  
  
btnClonar.addEventListener('click', () => {  
  if (numClones >= MAX_CLONES) return;  
  
  numClones++;  
  
  // Clonar el nodo con todos sus hijos (deep = true)  
  const clon = tarjetaOriginal.cloneNode(true);  
  
  // Eliminar el ID para evitar duplicados  
  clon.removeAttribute('id');  
  
  // Insertar al final del contenedor  
  contenedor.appendChild(clon);  
  
  if (numClones >= MAX_CLONES) {  
    btnClonar.disabled = true;  
  }  
});
```

Preguntas de validación:

1. ¿Qué método de JS permite clonar un nodo existente?

RTA// El método **elemento.cloneNode()**.

2. ¿Qué significa el parámetro booleano (true/false) en el método de clonación?

RTA// **true** realiza una clonación profunda, es decir, que copia el nodo y todo su contenido. **False** realiza una clonación superficial, solo copia la etiqueta de apertura y cierre, sin el contenido interior.

3. ¿Por qué es peligroso clonar elementos que tienen un ID único?

RTA// Porque al tener múltiples elementos con el mismo ID rompe la validación del HTML y hace que métodos como `getElementById` solo funcionen con el primero que encuentren.

4. ¿Cómo modificarías el nombre en la tarjeta clonada antes de insertarla?

RTA// Usando **querySelector** dentro del clon:

\$clon.querySelector('.nombre').textContent = "Nuevo Nombre";

5. ¿Qué método usas para insertar el clon al final de un contenedor específico?

RTA// El método **appendChild()**.

6. ¿Se clonan también los eventos (como clics) asociados al elemento original?

RTA// No, `cloneNode()` no copia los listeners asignados con `addEventListener()`. Solo se copian los atributos HTML como `onclick = "..."`.

7. ¿Cómo podrías limitar la cantidad de clones a un máximo de 5?

RTA// Validando la propiedad **.children.length** del contenedor dentro de un condicional `if` antes de ejecutar la clonación.

8. ¿Qué propiedad usarías para saber cuántos hijos tiene el contenedor de tarjetas?

RTA// La propiedad **.children.length** que cuenta solo elementos HTML o **.childrenNodes.length** que cuenta todo, incluyendo espacios en blanco y textos.

9. ¿Cómo podrías insertar el clon al principio de la lista en lugar de al final?

RTA// Usando el método **prepend()** en lugar de **appendChild()**.

10. ¿Qué diferencia hay entre clonar un nodo y simplemente crear uno nuevo desde cero?

RTA// Clonar es más rápido si la estructura es compleja, ya que se puede copiar todo el esqueleto HTML existente. En cambio, crear desde cero es más limpio si el elemento nuevo a ser muy diferente al original, pero este requiere más líneas de código para poder construir la estructura interna.